QUANTUM PIC 输入输出说明

当前显式 PIC 与隐式 PIC 模块的输入字典、输出文件、符号含义与输出机制

基于当前代码实现整理

2026-04-10

目录

1 文档目的与范围 **4**

2 整体输入文件体系 **4**

2.1 最小运行入口 4

2.2 关于 Units/Normalization/Gas/Reactions 的来源 5

2.3 显式与隐式模块的配置关系 5

3 输出目录总览 **5**

3.1 时序文件的重启行为 6

4 Output 字块的当前写法 **6**

4.1 推荐写法 6

4.2 兼容写法 8

4.3 Fields / Series / Console 的职责分工 8

4.4 当前 Console 输出存在的问题 8

4.5 Console 输出的目标重定义 9

4.6 建议的 Console 信息分层 9

4.6.1 1. Startup 一次性摘要 9

4.6.2 2. 周期性 Physics Headline 10

4.6.3 3. Debug / High-detail Drill-down 10

4.7 建议的默认终端内容重组 10

4.8 与 Series / Fields 的建议分工 11

4.9 建议的后续实现路线 11

5 通用输入项说明 **12**

5.1 PIC 主块 . . . 12

5.2 Units 与 Normalization . . . 12

5.3 Checkpoint . . . 12

6 字段快照文件与变量命名 **13**

6.1 字段快照的文件名 . . . 13

6.2 Fields/list 支持的常见变量 token . . . 13

7 显式模块 ExplicitPIC 的输入输出说明 **13**

7.1 输入关注点 . . . 13

7.2 显式模块输出目录结构 . . . 14

7.3 显式模块的字段与 profile 输出 . . . 14

7.3.1 `output/fields/fields_<step>.dat` . . . 14

7.3.2 `output/fields/profile1d.csv` . . . 14

7.3.3 `output/fields/profiles_avg.csv` . . . 15

7.4 显式模块时间序列输出 . . . 15

7.4.1 `output/series/timing.csv` . . . 15

7.4.2 `output/series/pic_diagnostics.csv` . . . 15

7.4.3 `output/series/power_step.csv` . . . 16

7.4.4 `output/series/rf_phase_report.csv` . . . 16

7.4.5 `output/series/current_waveform.csv` . . . 16

7.4.6 `output/series/current_balance.csv` . . . 17

7.4.7 `output/series/eb_band_rms.csv` . . . 17

7.4.8 `output/series/impact_report.csv` . . . 17

7.4.9 `output/series/particle_management_report.csv` . . . 17

7.4.10 `output/series/flux_report.csv` . . . 17

7.4.11 `output/series/particle_weight_stats.csv` . . . 18

7.4.12 `output/series/periodic_diagnostics.csv` . . . 18

7.4.13 统一物理诊断目录 `output/series/diagnostics` . . . 18

7.5 显式模块文字报告输出 . . . 19

8 隐式模块 ImplicitPIC 的输入输出说明 **19**

8.1 输入关注点 19
8.2 隐式模块字段输出 20
8.2.1 output/fields/fields_<step>.dat 20
8.2.2 output/fields/profile1d.csv 与 output/fields/profiles_avg.csv . . . 20
8.3 隐式模块时间序列输出 20
8.3.1 output/series/perf_step.csv 20
8.3.2 output/series/physics_step.csv 20
8.3.3 output/series/species_step.csv 21
8.3.4 output/series/amount_step.csv 与 output/series/amount_species_step.csv 21
8.3.5 output/series/coupling_fallbacks.csv 21
8.3.6 output/series/amr_adapt_step.csv 21
8.3.7 output/series/amr_levelset_diag.csv 21
8.3.8 output/series/boundary_flux_step.csv 22
8.3.9 output/series/current_balance.csv 22
8.3.10 output/series/geometry_metrics.csv 22
8.3.11 output/series/eb_band_rms.csv 22
8.3.12 output/series/current_waveform.csv 22
8.3.13 output/series/power_step.csv 23
8.3.14 统一 diagnostics 子目录 23
8.4 隐式模块元信息输出 23
8.4.1 output/meta/run_manifest.json 23
8.4.2 output/meta/ls_diag/ls_diag_*.dat 24

9 运行控制器写出的静态报告 24

10 符号、列名与单位约定 24
10.1 常见后缀 24
10.2 常见物理符号解释 24

11 如何理解这些输出 25

12 建议与注意事项 25

13 结语 26

1 文档目的与范围

本文面向当前 QUANTUM 中的两套主 PIC 模块：

- 显式模块 ExplicitPIC
- 隐式模块 ImplicitPIC

目标不是泛泛介绍 PIC 理论，而是把 “现在程序实际读什么、写什么、怎么配、文件里每一列是什么意思” 讲清楚，方便你从案例字典、运行输出和后处理文件三条线同时理解程序行为。

本文覆盖以下内容：

1. 运行时输入文件体系
2. `system` 目录中的主要字典及其职责
3. `Output` 字块的当前推荐写法与兼容写法
4. 显式与隐式模块各自的输出目录结构
5. 各输出文件的文件名、写出条件、主要列名与物理意义
6. 单位、符号、命名约定
7. 重启时序输出的当前行为

本文基于当前代码实现整理，不是旧设计文档，也不是未来计划稿。

2 整体输入文件体系

2.1 最小运行入口

一个标准案例通常从案例目录启动，关键输入位于 `system` 下。当前运行控制器默认允许的系统文件包括：

- `system/run.dict`
- `system/pic.dict`
- `system/mesh.dict`
- `system/geometry.dict`
- `system/mcc.dict`

其中：

文件	作用
`run.dict`	选择运行的 App。对当前 PIC 使用场景，通常是 `SingleModule`，再由运行控制器进入显式或隐式模块。
`pic.dict`	核心输入文件。包含时间步、步数、物种、边界条件、源项、输出、checkpoint、部分 Units 与 Normalization 信息。显式和隐式模块都读取它。
`mesh.dict`	背景网格、自适应细化规则、几何相关网格约束等。
`geometry.dict`	几何、边界 ID、相区、周期配对、MovingEB、motionSchedule 等。

文件	作用
mcc.dict	背景气体、截面、反应、碰撞模型等。运行控制器会从这里提取 Gas、CrossSections、Reactions、Collisions 等块。

2.2 关于 Units/Normalization/Gas/Reactions 的来源

当前实现允许两种来源：

- 新式来源：直接写在 pic.dict 或 mcc.dict 的对应块里
- 兼容来源:system/units.dict、system/normalization.dict、system/gas.dict、system/crossSections.d system/reactions.dict

但从当前案例演化趋势看，推荐优先使用：

- pic.dict 中的 Units 与 Normalization
- mcc.dict 中的 Gas/CrossSections/Reactions/Collisions 相关块

2.3 显式与隐式模块的配置关系

隐式模块的字典读取器 DictionaryImplicitPIC 本质上是对 DictionaryPIC 的薄封装，因此两套模块在很多输入语义上共用同一套 PicConfig 结构。这意味着：

- 大部分 Output 配置项在显式和隐式模块中名称一致
- 某些输出只有显式有，某些输出只有隐式有
- 同名输出的列定义在两模块内尽量对齐，但不保证完全一致

3 输出目录总览

当前输出已经按职能归类，不再建议把文件散落在案例根目录。总体上分为：

目录	含义
output/fields	场量快照与 profile 类输出。主要是随步号变化的场文件、1D profile、平均 profile。
output/series	时间序列输出。核心 CSV 都在这里，适合画时间历程、做守恒检查、诊断分析。
output/series/diagnostics	统一物理诊断子目录，当前包含电极电流、反应统计、能量直方图。基础路径可由 Output/Series/Diagnostics/baseDir 修改。
output/reports	主要给显式模块用的文字报告，内容偏事件或说明性文本，不是规范化时间序列。
output/meta	运行元信息与静态报告，例如 units、normalization、gas、cross section、reaction 报告，以及几何附属诊断。
output/meta/ls_diag	level set 诊断 ls_diag_*.dat 的归档位置。

目录	含义
cache 或配置指定路径	checkpoint 目录，不属于 output，但和输出强相关。

3.1 时序文件的重启行为

这是当前实现中的一个重要约定：

- 如果程序从 0 步冷启动，已有时序 CSV 会先清空，再从头写
- 如果程序从 checkpoint 重启，已有时序 CSV 中 step >= restartStep 的行会先被裁掉，只保留更早历史，再继续追加
- 这条规则适用于当前接入共享时序打开逻辑的 CSV 文件，即绝大多数 output/series 下的时间序列文件，以及统一 diagnostics writer 写出的 CSV
- output/fields 下按步命名的场快照文件现在也遵循同样的重启裁剪规则：0 步重启时清空，checkpoint 重启时删除 step >= restartStep 的旧快照后再继续写

因此现在不需要手工清理旧的时序 CSV 才能安全重启；程序会按 Checkpoint/restartStep 自动修整。

4 Output 字块的当前写法

4.1 推荐写法

当前推荐采用分组写法：

```
Output
{
    Fields
    {
        every 500
        list ( phi rho E n Te_e electron_mean_energy psi )

        Profile1D
        {
            enabled true
            nbinsX 128
        }

        Profile1DAvg
        {
            enabled true
            avgWindowSteps 1000
        }
    }
```

```
Series
{
    enabled true
    every 100

    CurrentWaveform { enabled true side auto }
    CurrentBalance  { enabled true }
    EBBandRMS       { enabled true bandLayers 1 }
    Amount          { enabled true }

    Diagnostics
    {
        enabled true
        every 100
        baseDir diagnostics

        ElectrodeCurrent
        {
            enabled true
            perBoundary true
            group anode 10
            group cathode 20 21 22
        }

        Reactions { enabled true }

        EnergyHistogram
        {
            enabled true
            bins 100
            maxEnergyEV 2000
        }
    }
}

Console
{
    level info
    every 100
    format multiline

    PeriodicDiagnostics
    {
        enabled true
        console true
```

```
        }

        Physics
        {
            enabled true
            detail medium
            showCharge true
            showCurrent true
            showWall true
            showMCC true
            showPerf true
        }
    }
}
```

4.2 兼容写法

当前解析器仍兼容多种扁平键写法，例如：

- `fields.every`
- `series.every`
- `currentwaveform.enabled`
- `diagnostics.energyhistogram.enabled`

但推荐逐步迁移到嵌套块写法，因为：

- 更接近实际输出目录的组织方式
- 更容易看出哪些选项属于 `Fields`、`Series`、`Console`
- 后续继续扩展时不容易把键名铺成一大片

4.3 Fields / Series / Console 的职责分工

分组	当前职责
`Fields`	决定场量快照、1D profile、平均 profile 等偏“空间分布”输出。
`Series`	决定各种 CSV 时间序列，包括守恒、功率、电流、反应、能量分布等。
`Console`	决定终端打印节奏与打印内容，不直接决定磁盘上的主 CSV 文件。

4.4 当前 Console 输出存在的问题

从当前 ExplicitPIC / ImplicitPIC 的实现看，`Console` 已经能打印 `Progress`、`Particles`、`Charge`、`Wall`、`Current`、`Energy`、`MCC`、`SEE`、`Embedded`、`Perf` 等段，但它还没有很好地满足“用户只看终端也能把握当前物理过程”这一目标。

当前主要问题可归纳为：

1. **职责偏移**：终端输出中混入了较多程序自检、实现细节和性能拆分信息，而物理过程主线不够突出。典型如 `Perf top=`、`fallbackFrac=`、大量 BC/geometry 映射与内部自检日志。
2. **实时判断所需的物理量不够集中**：用户最关心的通常是“现在放电/IEC 到什么阶段了、主电流和主损失走向怎样、是否已出现明显电离/SEE/壁面吸收、是否接近平衡”，但当前信息分散在多个段里，且很多关键物理量仍需要去 CSV 才能看明白。
3. **对嵌入边界问题不友好**：对于 IEC 这类多电极 embedded geometry，`Wall` 和 `Current` 仍以 `L/R/B/T` 域边界视角为主；这对平板/周期边界问题有用，但对用户实时判断阴极/阳极/导线组行为帮助有限。
4. **阶段感弱**：启动期、初始场建立、进入主推进、发生 AMR、输出落盘、checkpoint 写出，这些对用户很重要的“阶段事件”没有形成清晰统一的终端语义，因此容易把“启动很慢”误判成“程序卡住”。
5. **信息层级不清**：当前虽然有 `detail=minimal/medium/high` 和若干 `show*` 开关，但默认 `info` 级别下仍会看到不少更适合 `debug/high` 的内容，而真正适合日常盯跑的核心物理摘要没有被单独提升为“第一层”信息。
6. **与磁盘输出的分工还不够清楚**：终端里有些内容适合只看趋势，不适合堆细节；详细守恒、反应分项、功率分项、本应主要依赖 `series/*.csv`。当前 Console 还没有明确坚持“终端负责态势、CSV 负责细节”这一分工。
7. **格式可读性不足**：多组量仍采用长串拼接方式，带大量单位后缀、数组压缩表示和科学计数法，扫描成本高；终端最需要的是快速定位异常值和变化方向，而不是完整复刻表格式输出。

4.5 Console 输出的目标重定义

建议把 `Console` 的目标明确为：

> **用于在线跟踪当前计算的关键状态，帮助用户实时判断物理过程、运行阶段和是否异常；不承担完整报表职责。**

对应地，建议采用如下职责边界：

- `Console`：给出“现在发生了什么”、“是否正常”、“下一步该看哪里”。
- `Series`：记录可回放、可画图、可对比的完整时间序列细节。
- `Fields`：记录空间分布与场快照。

4.6 建议的 Console 信息分层

建议把终端输出分为三层：

4.6.1 1. Startup 一次性摘要

只在启动或关键重建后打印一次，突出用户真正需要确认的配置，而不是完整内部映射。

建议保留：

- 模块名、`dt`、`steps`、线程数、背景气体、物种列表

- 网格背景分辨率、是否启用 geometry/physics AMR、主要 cadence
- embedded geometry 总结：电极数量、boundaryId 分组、是否 moving EB
- 主要边界驱动：例如阴极/阳极或 RF 驱动是否存在

建议降级或折叠：

- 全部 `boundaryId -> BC` 明细
- 大段 level-set / cut-cell 自检日志
- 仅用于代码排查的 fallback / cache / predictor 内部提示

4.6.2 2. 周期性 Physics Headline

这是用户跑算例时最应该看到的默认层，建议固定收敛为少量高价值段：

段名	建议内容
`Progress`	`step/steps`、`t_s`、`dt_s`、`ETA`，并附带**阶段标签**，例如 `startup`、`field-init`、`transient`、`quasi-steady`、`amr-adapted`、`checkpoint-written`。
`State`	电势范围、电子平均能量/温度、代表性密度统计（如 n_e 均值/峰值或体平均/峰值），用于判断放电是否已建立、是否过热、是否接近稀薄/过密。
`Circuit`	以**物理电极组**而非单纯 `L/R/B/T` 为主，给出阳极/阴极/域边界净电流或净通量；IEC 类问题应优先显示 anode / cathode group。
`SourceSink`	本步或窗口内的主要来源与损失：注入、离化、SEE、domain loss、embedded loss、反应能量损失。用户应能直接看出“当前主要由谁驱动”。
`Health`	电荷平衡相对误差、必要时加上归一化残差或守恒告警，但只保留能帮助判断“算得是否可信”的指标。
`AMR`	仅在发生 adapt 或达到较粗 cadence 时打印：cells、$h_{\min}$、λ_D 约束最差比值、是否本步 adapt。

4.6.3 3. Debug / High-detail Drill-down

以下内容不宜长期占据默认 `info` 输出，应下放到 `detail=high` 或 `debug`：

- `Wall` 按 `L/R/B/T` 的全部分项
- `MCC` 的完整按通道/按 projectile 拆分
- `Embedded` 的按 boundaryId 明细
- `Perf top=`、Poisson fallback、cache miss、predictor 自检
- level-set / cut-cell / gather 权重等实现细节日志

4.7 建议的默认终端内容重组

当前代码已经按这一方向完成了第一阶段落地：默认 `info + detail=medium` 的周期性物理摘要收敛为 `Progress`、`State`、`Circuit`、`SourceSink`、`Health` 五段；`Particles`、`Wall`、`MCCSplit`、`Embedded`、`Perf` 等更细的实现/诊断信息下放到 `detail=high`。

进一步地，Progress 已从简单的 evt= 标签升级为带 stage= 的阶段语义，当前会按启动态、初始建场、过渡态和准稳态给出更明确的运行阶段判断;Circuit 在配置了 Diagnostics/ElectrodeCurrent/group 时会优先输出电极组电流视角，不再默认只停留在 L/R/B/T 域边界视角。

对日常盯跑场景，建议默认 info + detail=medium 输出只保留：

- Progress
- State
- Circuit
- SourceSink
- Health

其中：

- Perf 默认不打印，只在 debug 或显式 showPerf=true 且 detail=high 时打印。
- MCC 默认只保留一个 “总碰撞/总电离/总 SEE/主导通道” 摘要，不打印完整按通道展开。
- Wall 默认不打印域边界四侧宏粒子计数，除非算例本身就是平板/通道型、而不是电极组主导型。
- AMR 默认做事件式打印：只在 adapt 发生、网格规模明显变化、或用户显式请求时输出。

4.8 与 Series / Fields 的建议分工

为了避免终端信息继续膨胀，建议明确以下映射：

- 终端只看趋势和主导项;详细数值留给 overview_step.csv、power_step.csv、process_step.csv、species_budget_step.csv。
- 反应细分、按通道能量损失、按边界吸收功率，默认只在 CSV 里保留完整版本。
- 控制台若引用某个复杂量，最好让名字直接对应一个 CSV 主文件中的列族，方便用户进一步落盘分析。

4.9 建议的后续实现路线

建议分三步推进：

1. **先做信息减法**：把 Perf、大段 startup BC 明细、完整 MCC split 下放到 high/debug。
2. **再做物理主线重排**：把默认 info 收敛成 Progress/State/Circuit/SourceSink/Health 五段。
3. **最后做 case-aware 输出**：对 embedded electrode 类算例优先打印电极组电流/功率，对平板类算例保留 L/R/B/T 视图。

如果后续继续扩展配置,建议新增更贴近物理语义的开关,而不是继续堆叠 showCurrent/showWall/showMCC/sho
这类实现导向开关。例如：

- showState
- showCircuit
- showSourceSink
- showHealth

- showAmrEvents

这样可以让 Console 真正成为“在线态势面板”，而不是终端版的内部调试拼接字符串。

5 通用输入项说明

5.1 PIC 主块

PIC 块最基本的运行参数包括：

键	典型示例	含义
dt	2e-12 s	时间步长。最终会被换算到 code units 存储。
steps	1000000	总步数。
seed	12345	随机数种子。
omp	18	OpenMP 线程数提示。

5.2 Units 与 Normalization

这两块决定 code units 和 SI units 的换算关系。你在输出文件中经常会看到：

- time_code 与 time_SI_s 并存
- phi_code 与 phi_SI_V 并存
- Ex_code 与 Ex_SI_Vpm 并存

如果 writeSIOutputs=true，则不少显式输出会同时写 SI 列；隐式侧很多时间序列本身就直接使用 SI 物理量命名。

5.3 Checkpoint

Checkpoint 块控制缓存写出与重启：

键	示例	含义
enabled	true	是否写 checkpoint。
every	1000	每隔多少步写一次。
keep	3	仅保留最近多少个 step 目录。
path	"./cache"	checkpoint 根目录。
restart	true/false	是否从 checkpoint 重启。
restartStep	4 或 -1	指定重启步；负数通常表示取最新 checkpoint。
saveRng	true	是否保存 RNG 状态。

checkpoint 目录中通常会看到：

- cache/pic.<step>/pic_checkpoint.txt

- cache/pic.<step>/rng_state.txt
- 粒子缓存与网格缓存文件
- 隐式模块额外的 implicit_pic_fields_cache.bin

6 字段快照文件与变量命名

6.1 字段快照的文件名

显式和隐式模块都会把主场快照写成 Tecplot FEPOINT 格式：

- 显式:output/fields_000123.dat 旧路径已经迁移为概念上的 output/fields/fields_000123.dat
- 隐式：output/fields/fields_000123.dat

当前代码中,隐式模块明确使用 "fields/fields_" + tag 作为输出名;显式模块通过 fields_{tag} 的 Tecplot writer 生成快照，现行目录归类目标同样是 output/fields。

6.2 Fields/list 支持的常见变量 token

最常用的是：

token	含义
phi	电势 ϕ
rho	体电荷密度 ρ
E	电场向量，通常展开为 E_x 和 E_y
Ex, Ey	电场分量
n	物种数密度，通常展开为各物种 n_s
Te_e	电子温度，单位 eV
electron_mean_energy	电子平均能量，单位 eV
psi	level set / 几何函数 ψ
n_macro, w_sum, w_max	与宏粒子统计相关的诊断变量，主要在显式侧辅助调试与粒子管理分析

变量名在文件中可能出现 code-unit 列，也可能出现 SI 列，例如：

- phi_code / phi_SI_V
- Ex_code / Ex_SI_Vpm
- n_<species>_SI_1pm3

7 显式模块 ExplicitPIC 的输入输出说明

7.1 输入关注点

显式模块重点读取：

- `BC/Field` 与 `BC/Particle`
- `Species`
- `Deposition`
- `ParticleManagement`
- `SEE`
- `Output`
- `Checkpoint`

其中：

- `BC/Field` 控制电势边界与时间变 Dirichlet 波形
- `BC/Particle` 控制粒子边界模型以及 boundary-based SEE 配置
- `ParticleManagement` 控制 merge/split/thinning 与相关报表

7.2 显式模块输出目录结构

显式模块当前会使用：

- `output/fields`
- `output/series`
- `output/reports`
- `output/meta`

7.3 显式模块的字段与 profile 输出

7.3.1 `output/fields/fields_<step>.dat`

用途：主场快照，Tecplot FEPOINT 格式。

内容：

- 网格顶点坐标 x, y
- 由 `Output/Fields/list` 选择的场量
- 常见变量包括 ϕ, ρ, E_x, E_y, n_s, T_e, ψ

写出节奏：由 `Output/Fields/every` 控制。

7.3.2 `output/fields/profile1d.csv`

用途：沿 x 方向分箱、沿 y 平均的一维 profile，适合做 Turner 类 CCP benchmark 对比。

文件性质：长表结构，一个 `step` 下对应多个 `bin` 行。

典型内容：

- bin 编号与 x 坐标范围
- $\phi(x)$
- $E(x)$

- 电子/离子密度 $n_e(x)$、$n_i(x)$
- 电子温度 $T_e(x)$

配置入口：`Output/Fields/Profile1D`。

7.3.3 output/fields/profiles_avg.csv

用途：滑动窗口时间平均的一维 profile 快照。

这是覆盖式输出，不是追加式时间序列。每次写出时覆盖整份文件，表示 “当前滚动窗口的平均态”。

列定义按照 writer 注释，典型为：

- `bin`, `x_m`
- `phi_mean_V`, `phi_std_V`
- `E_mean_Vpm`, `E_std_Vpm`
- `ne_mean_m-3`, `ne_std_m-3`
- `ni_mean_m-3`, `ni_std_m-3`
- `Te_mean_eV`, `Te_std_eV`

配置入口：`Output/Fields/Profile1DAvg`。

7.4 显式模块时间序列输出

7.4.1 output/series/timing.csv

用途：每步耗时分解。

主要列：

- `step`, `time_s`
- `buildWeights_s`, `deposit_s`, `rhs_s`
- `poisson_assemble_s`, `poisson_factor_s`, `poisson_solve_s`, `poisson_total_s`
- `evalE_s`, `amr_indicator_s`, `amr_adapt_s`, `pm_s`, `gatherPush_s`, `output_s`, `total_s`
- `particles_total`, `poisson_N`, `poisson_nnz`, `poisson_usedSuperLU`

适合看性能瓶颈，而不是看物理量。

7.4.2 output/series/pic_diagnostics.csv

用途：显式 PIC 的标量摘要诊断。

它聚合了每一步的总体状态，例如：

- 活跃未知量规模
- 残差指标 r_G
- 场能量
- 总电荷与电荷不平衡
- 电子平均能量、电子温度

- 各物种宏粒子数与物理粒子数

这是显式模块总览性最强的 CSV 之一。

7.4.3 output/series/power_step.csv

用途：显式模块的全局能量与功率闭合诊断。

主要列：

- rG_L2, rG_Linf
- S_cf, S_eb 及其 p50/p90/p99 分位数
- field_energy_J, field_dWdt_W
- kinetic_e_J, kinetic_ion_J, kinetic_neu_J, kinetic_total_J
- mcc_energy_loss_J, see_emitted_energy_J, ion_impact_energy_J
- dE_drive_J, dE_inj_J, dE_abs_domain_J, dE_abs_embedded_J, dE_abs_total_J
- R_W_J, sum_phiI_W, sum_phiI_dt_J

这里的典型物理意义是：

- S_{cf}：粗细网格或复合带相关的闭合/匹配指标
- S_{eb}：嵌入边界附近的闭合/匹配指标
- field_dWdt_W：场能时间导数
- sum_phiI_W：电极驱动功率 $\sum \phi I$

7.4.4 output/series/rf_phase_report.csv

用途：记录时间变 Dirichlet 边界的瞬时相位与电压。

主要列：

- target：目标边界或对象
- type：波形类型
- freq_hz
- phase_rad, phase_deg
- phi_code, phi_SI_V

7.4.5 output/series/current_waveform.csv

用途：驱动电极电流波形。

列定义在 writer 中非常明确：

- step
- t_si_s
- V_drive_V
- J_cond_Am2
- J_disp_Am2

- `J_total_Am2 = J_cond + J_disp`

配置入口：`Output/Series/CurrentWaveform`，其中 `side` 可取 `left/right/bottom/top/auto`。

7.4.6 output/series/current_balance.csv

用途：显式模块的源/汇电流平衡检查。

主要列：

- 注入电流：`Ii_inj_A`, `Ie_inj_A`, `I_inj_mag_A`
- 流出电流：`I_out_domain_A`, `I_wall_eb_A`
- 平衡偏差：`deltaI_mag_A`, `deltaI_mag_rel`
- 带符号版本：`I_inj_signed_A`, `deltaI_signed_A` 等

7.4.7 output/series/eb_band_rms.csv

用途：嵌入边界附近电场强度的 RMS 代理量。

主要列：

- `band_layers`
- `cut_cells`, `band_cells`
- `E_rms_code`, `E_rms_Vpm`
- `E_mean_code`, `E_mean_Vpm`

7.4.8 output/series/impact_report.csv

用途：统计域边界粒子打到各侧面的宏粒子数。

主要按电子/离子区分，并按 left/right/bottom/top/unknown 列出。

7.4.9 output/series/particle_management_report.csv

用途：粒子管理 merge/split/thinning 过程的报表。

主要列：

- `kind`, `name`
- 管理前后宏粒子数、物理粒子数
- `merged_pairs`, `split_count`
- `spawn_in`, `spawn_kept`, `spawn_drop`
- `keep_prob`

7.4.10 output/series/flux_report.csv

用途：按边界侧面与物种统计流出通量和电流密度。

主要列：

- side, species
- macro_out, phys_out
- flux_macro_1pm2s, flux_phys_1pm2s
- current_Apm2

7.4.11 output/series/particle_weight_stats.csv

用途：每个物种宏粒子权重与单元占据统计。

主要列：

- n_macro, n_phys
- w_min, w_mean, w_p50, w_p90, w_p99, w_max
- cell_macro_max, cell_phys_max
- 权重最大占据单元的 cell index

7.4.12 output/series/periodic_diagnostics.csv

用途：周期边界闭合检查。

列：step,time_s,phiJumpX_V,phiJumpY_V,periodicX,periodicY。

7.4.13 统一物理诊断目录 output/series/diagnostics

当前包括三类：

电极电流 diagnostics 位于 output/series/diagnostics/electrodes。

文件：

- current_groups_step.csv
- current_boundaries_step.csv

组文件主要列：

- group
- boundary_count
- charge_C, current_A
- electron_charge_C, electron_current_A
- ion_charge_C, ion_current_A
- neutral_count_phys

这里的电流定义是单位时间内穿过指定电极边界组的净电荷通量。

反应 diagnostics 位于 output/series/diagnostics/reactions。

文件：

- `reaction_step.csv`
- `reaction_channel_step.csv`
- `reaction_projectile_step.csv`
- `reaction_spawn_step.csv`

典型物理量：

- `macro_collisions`, `phys_collisions`
- `collision_frequency_1ps`
- `macro_spawned`, `phys_spawned`
- `spawn_frequency_1ps`
- `energy_loss_J_total`
- `max_nu_dt_majorant`

其中 $\nu\Delta t$ majorant 是 MCC 采样中用于 majorant collision frequency 的稳定性/效率指标。

能量直方图 diagnostics 位于 `output/series/diagnostics/energy`。

当 `EnergyHistogram` 未显式写 `sample` 时，当前默认行为是：

- 对每个已配置物种自动生成一个 bulk-only histogram
- 默认文件名类似 `electron_bulk.csv`、`ar_plus_bulk.csv`

每个文件通常记录随时间演化的能量 bin 分布，用于观察电子、离子或 fast neutral 的能谱变化。

7.5 显式模块文字报告输出

这些文件在 `output/reports` 中，偏解释性，不是标准时序 CSV：

文件	内容
`collisions_report.txt`	碰撞统计与碰撞模型摘要。
`see_report.txt`	二次电子发射相关统计，包括离子撞击、发射能量等。
`ion_report.txt`	离子-中性碰撞减速与电荷交换相关统计。
`neutral_report.txt`	快中性粒子跟踪与能量统计。

8 隐式模块 ImplicitPIC 的输入输出说明

8.1 输入关注点

隐式模块与显式模块共享大部分 `pic.dict` 语义，但会额外使用：

- predictor/orbit-average 相关配置
- polarization Poisson 与隐式耦合参数
- AMR、自适应轨道积分、fallback 等诊断相关参数

- geometry 中的 motionSchedule 和 MovingEB 信息

8.2 隐式模块字段输出

8.2.1 output/fields/fields_<step>.dat

隐式模块同样输出 Tecplot FEPOINT 场快照，但在 cut-cell/EB 情况下支持 cell-constant packed output，以减少共享顶点重建带来的伪影。

这是理解隐式模块可视化结果时的关键区别：

- 普通模式：顶点共享 FEPOINT
- cut-cell 场景下可切换为 cell-constant packed FEPOINT

8.2.2 output/fields/profile1d.csv 与 output/fields/profiles_avg.csv

含义与显式模块基本一致，仍然是 1D 剖面和时间平均剖面。

8.3 隐式模块时间序列输出

8.3.1 output/series/perf_step.csv

这是隐式模块最重要的性能诊断文件。

主要列：

- np, total_s
- mcc_pre_s, buildWeightsN_s, predictorSegment_s, orbitAvgWeights_s, polPoisson_total_s
- AMR 相关耗时列
- is_output_step, is_amr_adapt_step, amr_skipped_by_gate
- pol_N, pol_nnz, pol_solver_mode
- predictor 与 orbit-average 的每粒子采样统计
- top1_name/top1_s 到 top3_name/top3_s

它直接告诉你隐式模块是卡在 polarization Poisson、orbit average、还是 predictor 分段上。

8.3.2 output/series/physics_step.csv

用途：隐式模块的主物理闭合摘要。

主要列：

- rG_L2, rG_Linf
- dQrel_stage, dQphys_rel
- Qsource_injected_C, Qsink_domain_C, Qsink_embedded_C
- Qmacro_begin_C, Qmacro_end_C
- field_energy_J, kinetic_e_J, kinetic_ion_J, kinetic_neu_J, total_energy_J

- J_cond_Apm2, J_disp_Apm2, J_total_Apm2
- macro_total, phys_total

8.3.3 output/series/species_step.csv

用途：按物种输出随时间变化的粒子统计。

每个物种都会自动展开成一组列：

- <species>_macro_count
- <species>_phys_count
- <species>_n_mean_1pm3
- <species>_mean_energy_eV
- <species>_Te_eV

8.3.4 output/series/amount_step.csv 与 output/series/amount_species_step.csv

仅当 Output/Series/Amount/enabled=true 且 amount cadence 有效时输出。

用途：检查 amount-formulated 量与实际物理粒子数之间的偏差。

8.3.5 output/series/coupling_fallbacks.csv

用途：统计权重构建与 orbit-average 过程中 coarse/fine fallback 的发生情况。

主要列：

- buildWeightsN_cwFallbackCount
- orbitAvg_sampleCwFallbackCount
- 对应的粒子数或 sample 数
- fallback 比例

这是定位隐式耦合近壁/AMR 区域退化路径的关键文件。

8.3.6 output/series/amr_adapt_step.csv

用途：AMR 适配动作本身的开销与结果。

主要列：

- cells_before, cells_after
- use_fast_remap
- mesh adapt、geometry refresh、face graph、embedded metrics、field rebuild 等耗时

8.3.7 output/series/amr_levelset_diag.csv

用途：每个 level set 的重初始化与数值状态诊断。

主要列：

- level_set_index
- moving, active_moving_levelset
- reinitialized, reinit_iterations
- reinit_fine_residual, reinit_coarse_residual
- min_value, max_value, near_zero_count, nonfinite_count, narrow_band_count

8.3.8 output/series/boundary_flux_step.csv

用途：与显式的 flux_report.csv 类似，但命名更直接，记录边界通量与电流密度。

8.3.9 output/series/current_balance.csv

用途：隐式模块的电荷/电流平衡。列为：

- Qsource_injected_C
- Qsink_domain_C
- Qsink_embedded_C
- Qmacro_begin_C, Qmacro_end_C
- dQmacro_step_C, dQphys_C, dQphys_rel

8.3.10 output/series/geometry_metrics.csv

只有在 motionSchedule 路径启用时输出。

用途：记录运动几何的结构参数与位移量，例如：

- stage, stage1_alpha, stage2_alpha
- gsa_mm, gad_mm, da_mm
- 加速栅/减速栅平移、侵蚀、上下游位置

8.3.11 output/series/eb_band_rms.csv

隐式侧版本较精简，主要列为：

- band_layers
- n_cells
- E_rms_Vpm
- E_mean_Vpm

8.3.12 output/series/current_waveform.csv

与显式侧共用同一个 writer，列含义完全相同：

- step

- t_si_s
- V_drive_V
- J_cond_Am2
- J_disp_Am2
- J_total_Am2

8.3.13 output/series/power_step.csv

隐式侧版本固定写 SI 尺度能量与功率列，主要包括：

- field_energy_J, field_dWdt_W
- kinetic_e_J, kinetic_ion_J, kinetic_neu_J, kinetic_total_J, kinetic_dWdt_W
- total_energy_J, total_dWdt_W
- dE_drive_J, dE_inj_J, dE_abs_domain_J, dE_abs_embedded_J, dE_abs_total_J
- R_W_J, sum_phiI_W, sum_phiI_dt_J

8.3.14 统一 diagnostics 子目录

隐式侧与显式侧共用：

- output/series/diagnostics/electrodes
- output/series/diagnostics/reactions
- output/series/diagnostics/energy

列定义与显式侧相同，因为 writer 是共享的。

8.4 隐式模块元信息输出

8.4.1 output/meta/run_manifest.json

这是隐式模块当前独有的结构化运行清单。

它记录：

- 模块名、case 路径、字典路径
- output.root、fieldsDir、seriesDir、metaDir
- cadence 配置
- units 标度
- normalization 信息
- species 列表
- field token 到 Tecplot 变量名的映射
- 实际选择的字段 token

如果你要做自动后处理，这个文件非常有用。

8.4.2 output/meta/ls_diag/ls_diag_*.dat

这是几何 level set 诊断文件，由几何实现层写出，不专属于显式或隐式。

文件名格式大致为：

- ls_diag_LS<levelSetI>_<tag>_c<callIdx>.dat

用于排查 level set、组合几何、等值面演化等问题。

9 运行控制器写出的静态报告

无论显式还是隐式，只要运行控制器成功解析对应对象，通常都会在 output/meta 下写一些静态说明文件：

文件	内容
units_report.txt	当前单位制、基本标度、是否写 SI 输出等。
normalization_report.txt	Normalization 策略、来源、dimension、slab thickness 等。
gas_report.txt	背景气体名称、压强、温度、数密度、热速等。
crossSections_report.txt	截面数据库摘要。
reactions_report.txt	反应表摘要。

这些文件是 “静态背景说明”，不是时间历程。

10 符号、列名与单位约定

10.1 常见后缀

后缀	含义
_code	code units 下的量。
_SI_s / _SI_V / _SI_Vpm	SI 单位下的时间、电势、电场。
_1pm3	每立方米，m^{-3}。
_Apm2	安培每平方米，$\mathrm{A/m}^2$。
_J	焦耳。
_W	瓦特。
_eV	电子伏特。
_phys	物理粒子数或物理量加权总量。
_macro	宏粒子数或宏粒子尺度量。

10.2 常见物理符号解释

符号/列	含义
ϕ	电势。
ρ	电荷密度。
$\mathbf{E}$	电场向量。
n_s	第 s 个物种的数密度。
T_e	电子温度，当前输出中通常用 eV 表示。
r_G	泊松或耦合闭合相关的残差指标，常见 `rG_L2` 和 `rG_Linf`。
S_{cf}	coarse-fine 接口/复合带相关诊断量。
S_{eb}	embedded boundary 附近诊断量。
J_{cond}	传导电流密度。
J_{disp}	位移电流密度。
J_{total}	总电流密度，$J_{cond} + J_{disp}$。
$\sum \phi I$	电极驱动功率项。
$\nu \Delta t$	碰撞 majorant 频率与时间步的乘积，反映 MCC 采样强度。

11 如何理解这些输出

如果你想快速读懂一个算例，我建议按下面顺序看：

1. 先看 `output/meta` 里的 `units_report.txt`、`normalization_report.txt`，确认单位和归一化
2. 看 `Output` 字典，确认 fields cadence 和 series cadence
3. 看 `output/series` 中总览性文件
   - 显式先看 `pic_diagnostics.csv`、`power_step.csv`、`timing.csv`
   - 隐式先看 `physics_step.csv`、`perf_step.csv`、`power_step.csv`
4. 如果关心边界电流和驱动响应，再看 `current_waveform.csv`、`current_balance.csv` 和 diagnostics/electrodes
5. 如果关心碰撞化学，再看 diagnostics/reactions
6. 如果关心能谱，再看 diagnostics/energy
7. 最后用 `output/fields` 中的场快照与 1D profile 理解空间结构

12 建议与注意事项

- `EnergyHistogram` 现在不需要手工写一堆 `sample`，缺省就是每个物种一个 bulk histogram
- 时序 CSV 现在支持 restart-aware 裁剪，不要再手工拼接不同重启阶段的旧文件
- 显式与隐式的字段快照名字相近，但隐式在 cut-cell/EB 场景下更可能使用 cell-constant packed 输出
- 如果你做自动后处理，隐式模块优先读取 `output/meta/run_manifest.json`
- 如果输出看起来很多，先从总览文件入手，不要一开始就逐个看所有 CSV

13 结语

当前 QUANTUM 的 PIC 输入输出体系已经不再是单一 “写几个散乱 dat 文件” 的模式，而是明确分成：

- 空间快照 `fields`
- 时间序列 `series`
- 说明性文本 `reports`
- 静态元信息 `meta`

如果你后续要继续整理，我建议下一步做两件事：

1. 给每类输出再补一份最小后处理脚本入口
2. 把最常用的 5 到 8 个 CSV 做成统一 plotting notebook 或 Python 工具

这样你以后看一个新 case，就能先通过元信息和总览序列迅速定位，再下钻到场快照与反应/能谱细节。